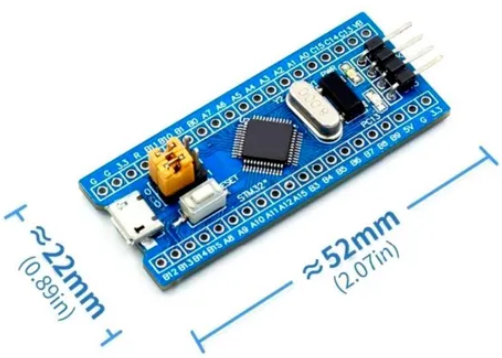
1. Introducción

En el desarrollo de sistemas embebidos existen distintos niveles de abstracción: desde entornos que ocultan los detalles del hardware para facilitar el aprendizaje, hasta la programación bare metal, donde el programador interactúa directamente con los registros del microcontrolador sin intermediarios. Esta guía propone trabajar en ese nivel más bajo, utilizando únicamente herramientas estándar del ecosistema GNU, con el objetivo de comprender en profundidad cómo funciona el hardware y qué ocurre realmente desde que se enciende el microcontrolador hasta que se ejecuta la primera instrucción del programa. A continuación se detalla el hardware y software utilizado para el desarrollo del trabajo práctico.

1.1. Hardware

- **Placa BluePill**: esta placa posee el microcontrolador **STM32F103C8T6**, con núcleo ARM Cortex-M3, de bajo costo (principalmente en su versión clon). Figura 1a.
  - Frecuencia de reloj: 72 MHz.
  - Memoria: 64 KB de memoria Flash y 20 KB de RAM.
  - Periféricos: 37 pines GPIO, 2 ADC de 12 bits, 2 interfaces I2C, 3 interfaces UART, 2 interfaces SPI, 1 interfaz CAN, 7 temporizadores.
  - Compatibilidad: Compatible con Arduino IDE, Keil MDK y CubeMX.
- **Programador ST-Link V2**: este programador permite conectarse a la placa no solamente para poder grabar la aplicación, sino además permite el depurado (debug) de la misma. Figura 1b.



(a) Placa BluePill.

(b) Programador ST-Link V2.

Figura 1: Hardware del TP1.

El pinout de la placa incluye detalles importantes:

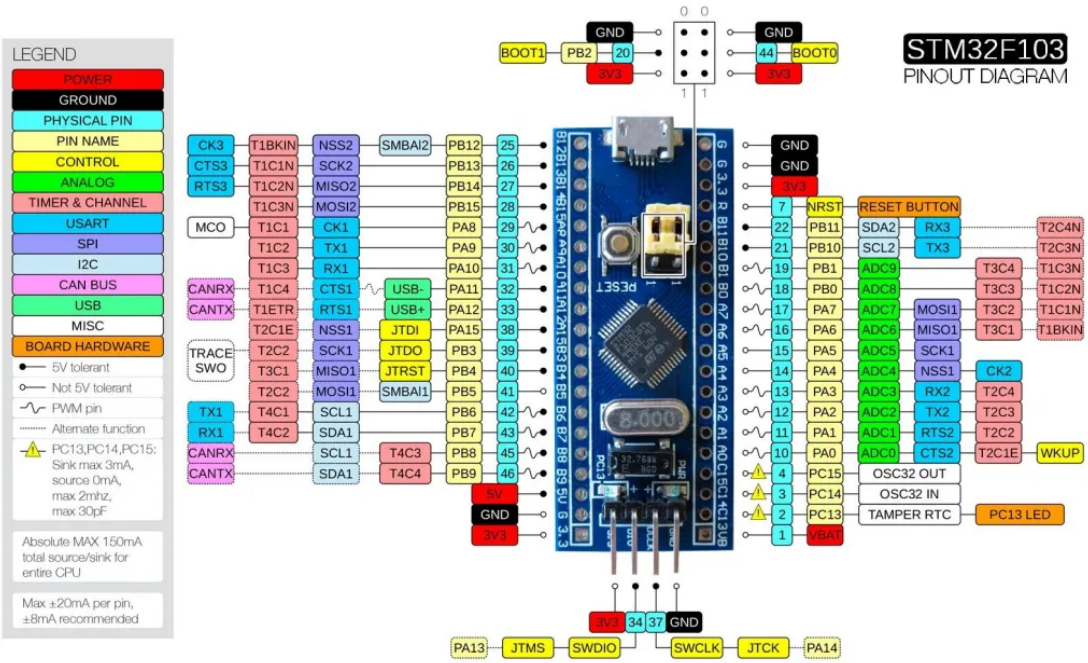
1. **Tolerancia a 5 V:** Se pueden observar los pines que son tolerantes a 5 V y aquellos que solo soportan hasta 3,3 V.

2. **Alimentación:** Los recuadros en rojo indican las diferentes entradas de alimentación disponibles y las tensiones; en la mayoría puede ser de 3,3 V y una entrada de 5 V; además se puede alimentar con 5 V directamente del USB.
   En todo momento es importante alimentar la placa desde un solo punto; si alimentamos de dos lados, hay que tener en cuenta que estaremos puenteando las dos fuentes.
   La entrada de 5 V (tanto por el USB como por el pin de 5 V) pasa por un regulador a 3,3 V.
   Un caso especial es la entrada `VBAT`, utilizada para alimentar el RTC (Real-Time Clock) interno del microcontrolador.

3. **Salida para pruebas:** La única salida disponible sin requerir hardware extra es el LED conectado al pin PC13.

4. **Botón de Reset:** Permite el reinicio de la aplicación.

5. **Conexión a dispositivo para grabar/depurar:** Descripción de los pines JTAG para realizar la conexión con el grabador ST-Link V2.



1.2. Software

Las herramientas que utilizaremos permiten generar los binarios de nuestra aplicación. Como la escritura y compilación se realizará en una computadora personal (generalmente en arquitectura x86) para que finalmente corra en una arquitectura ARM, el proceso de compilación será **cruzado** (cross compilation), es decir, compilamos en una arquitectura para que funcione en otra.

Por ello, los compiladores normalmente instalados en la PC no funcionarán y deberemos instalar los siguientes:

- **GNU ARM Toolchain:** este paquete de herramientas permite realizar todos los procesos necesarios para transformar el código escrito en texto plano al binario final para grabar a la placa.
  Paquete en Linux: `gcc-arm-none-eabi`
- **Make:** herramienta utilizada para la administración del proyecto.
  Paquete en Linux: `make`
- **OpenOCD:** herramienta que permite grabar la aplicación a la placa y luego depurarla.
  Paquete en Linux: `openocd`
- **GDB:** herramienta de depuración. Generalmente está instalada en su versión solo x86, por eso deberemos instalar la versión multiarquitectura.
  Paquete en Linux: `gdb-multiarch`

Instalación de las herramientas bajo Linux (Ubuntu):

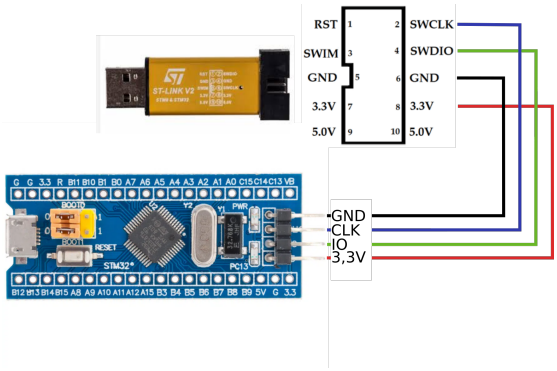
```
$ sudo apt install gcc-arm-none-eabi make openocd gdb-multiarch
```

Conexión del Hardware

El programador **ST-Link V2** posee un puerto USB que se conecta directamente a la computadora donde se realizará el proyecto.

La conexión que requiere mayor atención es aquella entre el programador y la placa, para lo cual se utilizan los 4 cables que trae la placa programadora. Estos cuatro cables se conectan al puerto JTAG del microcontrolador.

Importante: Hay programadores que poseen distribuciones de pines diferentes; prestar mucha atención al plano con el pinout que figura en el propio ST-Link V2 para verificar que coincide con el mostrado.



Una vez conectado, se puede verificar la conexión de la placa y el correcto funcionamiento del grabador con:

```
$ openocd -f interface/stlink.cfg -f target/stm32f1x.cfg
```

Si funciona correctamente, mostrará información de la placa.

Placa Clon

Una de las tareas del OpenOCD es verificar el ID del microcontrolador para determinar si la configuración es la correcta. Algunas placas clon poseen otro identificador, con lo cual esta verificación falla.

En caso de no funcionar el OpenOCD, se puede intentar la siguiente configuración para renombrar el ID con un posible ID de las placas clon:

```
$ openocd -f interface/stlink.cfg -c "set CPUTAPID 0x2ba01477" -f target/
  stm32f1x.cfg
```

2. Proyecto de Blink LED

El primer proyecto realizará un blink (parpadeo) en el único LED disponible en nuestra placa BluePill. Para esto, necesitaremos una serie de archivos que formarán nuestro primer proyecto.

Los archivos pueden ser bajados desde la UV de la cátedra.

- **main.c**: En este archivo escribiremos nuestra función principal `main()`, que realizará un bucle infinito encendiendo y apagando el LED.

- **crt.s**: Este archivo no es común en las aplicaciones que funcionan en un SO como Linux o Windows, pero adquiere gran importancia en las aplicaciones que corren en microcontroladores sin SO previo a la ejecución de la aplicación.
  El archivo `crt.s`, denominado comúnmente *startup*, es el encargado de realizar las configuraciones críticas del hardware para finalmente entregar el mando al `main()` del `main.c`.
- **Makefile**: Archivo de configuración para que `make` automatice la gestión de la aplicación.
- **linker.ld**: Al igual que `crt.s`, no es común en aplicaciones con SO.
  Este archivo configura al enlazador (linker) según las especificaciones particulares del microcontrolador: tamaño de memoria, disposición de la memoria, etc.

2.1. Aplicación (`main.c`)

Este archivo cumple la función de nuestra aplicación: realizar un parpadeo en el LED disponible en la placa.

Nota: En los siguientes párrafos se detalla cómo configurar la GPIO para acceder al LED; es conveniente haber entendido previamente la configuración de este periférico y los registros asociados.

En primer lugar, necesitamos identificar la GPIO que controla el LED. Observando el plano, identificamos `PC13` como el pin conectado al LED, es decir, el pin 13 del puerto C.

Habilitación y configuración del puerto C del GPIO:

- Como es el puerto C, habilitamos este puerto con un `1` en el bit 4 del registro `RCC_APB2ENR`.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved										TIM11 EN	TIM10 EN	TIM9 EN	Reserved		
										rw	rw	rw			

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADC3 EN	USART EN	TIM8 EN	SPI1 EN	TIM1 EN	ADC2 EN	ADC1 EN	IOPG EN	IOPF EN	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.	AFIO EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw

Las siguiente línea de código habilita el puerto C:

```
RCC_APB2ENR |= 1 << 4; //Enable GPIOC Clock
```

- Luego configuramos la `GPIOC13` con los registros `GPIOC_CRL/H`:

- Por ser el bit 13, utilizamos el registro de la parte alta (`GPIOC_CRH`) que se detalla en la siguiente tabla.
- Luego $13 - 8 = 5$, es el grupo 5 del CRH, ubicado a partir del bit 20 (resaltado en rojo).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- Ubicados los dos campos para configurar el pin, procedemos a establecer el modo de funcionamiento:
  - `MODE = b10` → salida con un slew rate de 2 MHz
  - `CNF = b00` → modo push-pull

  Finalmente los 4 bits quedan como `b0010` → `0x2`.

Las siguientes líneas del código configuran el modo de funcionamiento de PC13:

```
GPIOC_CRH &= 0xFF0FFFFF;
GPIOC_CRH |= 0x00200000;
```

Encendido y apagado del LED conectado al PC13:
Configurado el puerto, el resto del código se encarga de realizar un bucle infinito, estableciendo en alto el PC13:

```
GPIOC_ODR |= GPIOC13;
```

Y luego de un retardo, pone en bajo el mismo bit:

```
GPIOC_ODR &= ~GPIOC13;
```

Y luego de un nuevo retardo el ciclo se repite.
A continuación se detalla el código completo de la aplicación **main.c**.

main.c

```c
#include <stdint.h>
// register address
#define RCC_BASE    0x40021000
#define GPIOC_BASE  0x40011000
#define RCC_APB2ENR *(volatile uint32_t *)(RCC_BASE   + 0x18)
#define GPIOC_CRH   *(volatile uint32_t *)(GPIOC_BASE + 0x04)
#define GPIOC_ODR   *(volatile uint32_t *)(GPIOC_BASE + 0x0C)
// bit fields
#define RCC_IOPCEN (1 << 4)
#define GPIOC13    (1UL << 13)

void main(void)
{
    RCC_APB2ENR |= RCC_IOPCEN;
    GPIOC_CRH &= 0xFF0FFFFF;
    GPIOC_CRH |= 0x00200000;
    while (1)
    {
        GPIOC_ODR |= GPIOC13;
        for (int i = 0; i < 500000; i++); // arbitrary delay
        GPIOC_ODR &= ~GPIOC13;
        for (int i = 0; i < 50000; i++);  // arbitrary delay
    }
}
```

2.2. Startup (`crt.s`)

Este archivo en ensamblador, denominado *startup*, contiene la secuencia de arranque del microcontrolador. Posee los valores correspondientes a los vectores de interrupción, inicializa variables, inicializa el stack, configura hardware crítico y finalmente entrega el mando al `main()`.

En este caso se muestra la versión más sencilla del startup: solamente inicializa el stack y vincula el `main()` con el reset del microprocesador.

Como detalle importante se puede ver:

```
.word _esstack
.word reset
```

Estos `word` que se grabarán al inicio de la memoria, y corresponden en primer lugar al puntero del stack (su valor se obtiene del archivo denominado linkerscript). Y como segundo word el denominado

vector de reset, donde se graba la dirección de inicio de la aplicación.

Estos `word` son requerimiento estricto del hardware de este microcontrolador. A continuación del vector de reset continúan otros vectores para otras excepciones que no se detallan en este momento por no ser requeridas.

`crt.s`

```
.syntax unified
.cpu cortex-m3
.thumb
.global _esstack
.word _esstack
.word _reset
.thumb_func
_reset:
    bl main
    b .
```

2.3. LinkerScript (`linker.ld`)

El linkerscript es un archivo de texto que indica al enlazador (linker) cómo los diferentes archivos objetos y librerías deben ser combinados. Define las posiciones de la memoria de variables y de programa, y la posición de la primera instrucción, entre otras cosas. En su versión más simple:

1. Se define la sección de FLASH (código) con el origen y su tamaño.
2. Se define de la misma manera la RAM.
3. Mediante una variable global que luego puede ser leída por el startup, se define el puntero del stack pointer (`_esstack`) como la posición más alta de la memoria RAM.

`linker.ld`

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
    RAM (xrw)  : ORIGIN = 0x20000000, LENGTH = 20K
}
_esstack = ORIGIN(RAM) + LENGTH(RAM);
```

2.4. Makefile

El `Makefile` automatiza la compilación y el grabado del proyecto utilizando la herramienta `make`.

Makefile

```
SRC_DIR    = src
BUILD_DIR  = build
CC         = arm-none-eabi-gcc
AS         = arm-none-eabi-as
LD         = arm-none-eabi-ld
BIN        = arm-none-eabi-objcopy
CFLAGS     = -mthumb -mcpu=cortex-m3 -O0 -g

all: app.bin

$(BUILD_DIR):
        mkdir $(BUILD_DIR)

crt.o: $(BUILD_DIR) $(SRC_DIR)/crt.s
        $(AS) -o $(BUILD_DIR)/crt.o $(SRC_DIR)/crt.s

main.o: $(BUILD_DIR) $(SRC_DIR)/main.c
        $(CC) $(CFLAGS) -c -o $(BUILD_DIR)/main.o $(SRC_DIR)/main.c

app.elf: $(BUILD_DIR) $(SRC_DIR)/linker.ld crt.o main.o
        $(LD) -T $(SRC_DIR)/linker.ld -o $(BUILD_DIR)/app.elf \
        $(BUILD_DIR)/crt.o $(BUILD_DIR)/main.o

app.bin: app.elf
        $(BIN) -O binary $(BUILD_DIR)/app.elf $(BUILD_DIR)/app.bin

clean:
        rm -rf $(BUILD_DIR)

openocd-flash: app.bin
        openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \
        -c "init" -c "reset halt" \
        -c "flash write_image erase $(BUILD_DIR)/app.bin 0x08000000" \
        -c "reset run" -c "exit"

openocd-erase:
        openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \
        -c "init" -c "reset halt" \
```

```
        -c "flash erase_sector 0 0 last" -c "reset run" -c "exit"
```

2.5. Depuración

Una vez compilado con:

```
$ make
```

y grabado a la placa con:

```
$ make openocd-flash
```

podemos pasar a depurar la aplicación utilizando `gdb-multiarch`.

La depuración se realiza en dos instancias:

1. Conexión con OpenOCD:

```
$ openocd -f interface/stlink.cfg -f target/stm32f1x.cfg
```

(Recordá siempre agregar `set CPUTAPID 0x2ba01477` si es una placa clon.)

2. Lanzar GDB en otra terminal:

```
$ gdb-multiarch build/app.elf
```

Ya dentro de GDB, conectarse al puerto que abrió OpenOCD:

```
(gdb) target extended-remote :3333
```

Comandos útiles de GDB

- `break {linea}` — genera un breakpoint en la línea indicada.
- `continue` — continúa la ejecución hasta el próximo breakpoint.
- `list` — lista el código fuente.
- `step` — salta a la próxima línea (entrando en funciones).
- `next` — salta a la próxima línea de la función actual.
- `stepi` y `nexti` — hacen lo mismo que `step`/`next` pero a nivel de línea en ensamblador.

Para abrir ventanas de visualización:

- `layout src` — muestra el código fuente y el cursor.
- `layout regs` — muestra los registros.
- `layout asm` — muestra el ensamblador.